

Title of Topic: OPERATORS IN SQL

Course Code: 23IT201

Course Title: DataBase Management Systems

MODULE –II

Academic Year: 2024 - 2025 (Even Sem)

Addition Operator

returns new column named total_amount which is 100 added to the amount field

```
SELECT item, amount, amount+100 AS total_amount FROM Orders;
```

SUBTRACTION

- returns new column named offer_price which is
- 20 subtracted to the amount field

```
SELECT item, amount, amount-20 AS offer_price  
FROM Orders;
```

MULTIPLICATION

- returns new column named total_amount which is**
- 4 multiplied to the amount field**

```
SELECT item, amount, amount*4 AS total_amount  
FROM Orders;
```

DIVISION

- returns new column named half_amount which is**
- divided by 2 to the amount field**

```
SELECT item, amount, amount/2 AS half_amount  
FROM Orders;
```

MODULO

-- returns 1 which is remainder

SELECT 10 % 3 AS result;

EQUAL TO

-- returns records where customer_id is only 4

```
SELECT order_id, item, amount
```

```
FROM Orders
```

```
WHERE customer_id = 4;
```

LESS THAN

-- returns records where amount is less than 400 (exclusive)

```
SELECT order_id, item, amount
```

```
FROM Orders
```

```
WHERE amount < 400;
```


GREATER THAN

-- returns records where amount is greater than 400 (exclusive)

```
SELECT order_id, item, amount
```

```
FROM Orders
```

```
WHERE amount > 400;
```

LESS THAN EQUAL TO

-- returns records where amount is less than or equal to 400

```
SELECT order_id, item, amount
```

```
FROM Orders
```

```
WHERE amount <= 400;
```

GREATER THAN EQUAL TO

-- returns records where amount is greater than or equal to 400

```
SELECT order_id, item, amount
```

```
FROM Orders
```

```
WHERE amount >= 400;
```

NOT EQUAL

-- returns records where amount is not equal to 400

```
SELECT order_id, item, amount
```

```
FROM Orders
```

```
WHERE amount != 400;
```

```
SELECT order_id, item, amount
```

```
FROM Orders
```

```
WHERE amount <> 400;
```

ANY

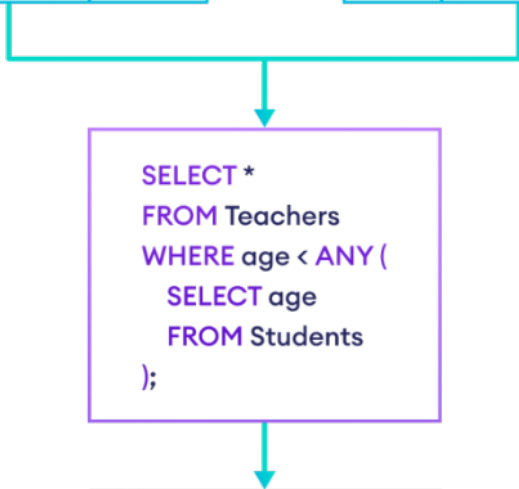
```
SELECT *  
FROM Teachers  
WHERE age = ANY (  
    SELECT age  
    FROM Students  
);
```

Table: Teachers

id	name	age
1	Peter	32
2	Megan	43
3	Rose	29
4	Linda	30
5	Mary	41

Table: Students

id	name	age
1	Harry	23
2	Jack	42
3	Joe	32
4	Dent	23
5	Bruce	40



```
SELECT *  
FROM Teachers  
WHERE age < ANY (  
  SELECT age  
  FROM Students  
);
```

id	name	age
1	Peter	32
3	Rose	29
4	Linda	30
5	Mary	41

ALL

```
SELECT *
```

```
FROM Teachers
```

```
WHERE age > ALL (
```

```
    SELECT age
```

```
    FROM Students
```

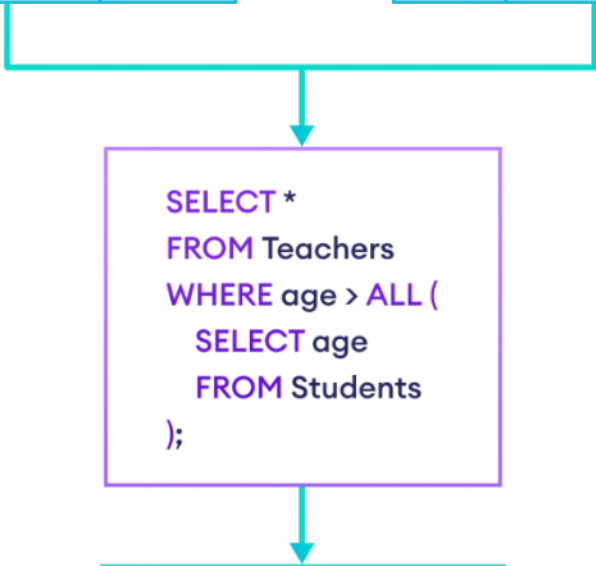
```
);
```

Table: Teachers

id	name	age
1	Peter	32
2	Megan	43
3	Rose	29
4	Linda	30
5	Mary	41

Table: Students

id	name	age
1	Harry	23
2	Jack	42
3	Joe	32
4	Dent	23
5	Bruce	40



```
SELECT *  
FROM Teachers  
WHERE age > ALL (  
  SELECT age  
  FROM Students  
);
```

id	name	age
2	Megan	43

AND

**-- select the first_name and last_name of all customers
-- who live in 'USA' and have the last name 'Doe'**

```
SELECT first_name, last_name  
FROM Customers  
WHERE country = 'USA' AND last_name = 'Doe';
```

OR

**-- select first and last name of customers
-- who either live in the USA
-- or have the last name 'Doe'**

```
SELECT first_name, last_name  
FROM Customers  
WHERE country = 'USA' OR last_name = 'Doe';
```

NOT

-- select customers who don't live in the USA

```
SELECT first_name, last_name  
FROM Customers  
WHERE NOT country = 'USA';
```

MULTIPLE OPERATORS

-- select customers who live in either USA or UK and whose age is less than 26

SELECT *

FROM Customers

WHERE (country = 'USA' OR country = 'UK') AND age < 26;

MULTIPLE OPERATORS

-- exclude customers who are from the USA and have 'Doe' as their last name

SELECT *

FROM customers

WHERE NOT country = 'USA' AND NOT last_name = 'Doe';

BETWEEN

-- select rows where the amount is between 200 and 600

SELECT item, amount

FROM Orders

WHERE amount BETWEEN 200 AND 600;

IN

-- select customers from the USA

```
SELECT first_name, country  
FROM Customers  
WHERE country IN ('USA');
```

NOT IN

-- select rows where country is not UK or UAE

```
SELECT first_name, country  
FROM Customers  
WHERE country NOT IN ('UK', 'UAE');
```


EXISTS

- The EXISTS operator is used to test for the existence of any record in a subquery.
- The EXISTS operator returns TRUE if the subquery returns one or more records.

EXISTS

To fetch the first and last name of the customers who placed atleast one order.

Orders

order_id	c_id	order_date
1	407	2017-03-03
2	405	2017-03-05
3	408	2017-01-18
4	404	2017-02-05

Customers

customer_id	lname	fname	website
401	Singh	Dolly	abc.com
402	Chauhan	Anuj	def.com
403	Kumar	Niteesh	ghi.com
404	Gupta	Shubham	jkl.com
405	Walecha	Divya	abc.com
406	Jain	Sandeep	jkl.com
407	Mehta	Rajiv	abc.com
408	Mehra	Anand	abc.com

EXISTS

SELECT fname, lname

FROM Customers

WHERE EXISTS (SELECT * FROM Orders

WHERE Customers.customer_id = Orders.c_id);

fname	lname
Shubham	Gupta
Divya	Walecha
Rajiv	Mehta
Anand	Mehra

NOT EXISTS

Fetch last and first name of the customers who has not placed any order.

```
SELECT lname, fname
```

```
FROM Customers
```

```
WHERE NOT EXISTS (SELECT * FROM Orders WHERE  
Customers.customer_id = Orders.c_id);
```

Using EXISTS condition with DELETE statement

Delete the record of all the customer from Order Table whose last name is 'Mehra'.

```
DELETE
```

```
FROM Orders
```

```
WHERE EXISTS (SELECT *
```

```
    FROM customers
```

```
    WHERE Customers.customer_id = Orders.c_id
```

```
    AND Customers.lname = 'Mehra');
```

When to Use SQL EXISTS

- The EXISTS condition is particularly useful in the following scenarios:
- **Checking Data Existence:** You can use **EXISTS** to check if related data exists in another table before performing an action (e.g., selecting, updating, deleting).
- **Performance:** EXISTS is often more efficient than using IN when dealing with large **datasets**, as it stops searching once a match is found.
- **Correlated Subqueries:** EXISTS is ideal for correlated **subqueries**, where the subquery refers to the outer query's values.

Differences Between EXISTS and IN

- EXISTS is used for checking the existence of rows, while **IN** checks if a value matches any value from a list or **subquery** result.
- EXISTS is more efficient when the subquery results in a large number of rows.
- IN works well for **small datasets** or static lists of values.

Using EXISTS condition with UPDATE statement

Update the Iname as 'Kumari' of customer in Customer Table whose customer_id is 401.

```
UPDATE Customers  
SET Iname = 'Kumari'  
WHERE EXISTS (SELECT *  
              FROM Customers  
              WHERE customer_id = 401);
```


LIKE

```
SELECT *
```

```
FROM Customers
```

```
WHERE country LIKE 'U_';
```

selects customers whose country name starts with U and is followed by only one character.

LIKE

SELECT *

FROM Customers

WHERE country NOT LIKE 'USA';

LIKE

```
SELECT *
```

```
FROM Customers
```

```
WHERE last_name LIKE 'R%t' OR last_name LIKE '%e';
```

IS NULL

```
SELECT *  
FROM Employee  
WHERE email IS NULL;
```

SET OPERATORS

- There are the following 4 set operators in SQL Server:
- **UNION**: Combine two or more result sets into a single set without duplicates.
- **UNION ALL**: Combine two or more result sets into a single set, including all duplicates.
- **INTERSECT**: Takes the data from both result sets, which are in common.
- **EXCEPT**: Takes the data from the first result set but not in the second result set (i.e., no matching to each other)

SET OPERATORS

Rules on Set Operations:

- The result sets of all queries must have the same number of columns.
- In every result set, the data type of each column must be compatible (well-matched) with the data type of its corresponding column in other result sets.
- An ORDER BY clause should be part of the last select statement to sort the result. The first select statement must find out the column names or aliases.

UNION Operator

- The Union operator will return all the unique rows from both queries. Notice that the duplicates are removed from the result set.

SELECT ID, Name, Gender, Department FROM TableA

UNION

SELECT ID, Name, Gender, Department FROM TableB

SELECT * FROM TableA

ID	Name	Gender	Department
1	Pranaya	Male	IT
2	Priyanka	Female	IT
3	Preety	Female	HR
3	Preety	Female	HR

SELECT * FROM TableB

ID	Name	Gender	Department
2	Priyanka	Female	IT
3	Preety	Female	HR
4	Anurag	Male	IT

Result:

ID	Name	Gender	Department
1	Pranaya	Male	IT
2	Priyanka	Female	IT
3	Preety	Female	HR
4	Anurag	Male	IT

UNION Operator

- **Purpose:** The UNION operator combines the result sets of two or more SELECT statements into a single result set.
- **Distinct Values:** It removes duplicate rows between the various SELECT statements.
- **Use Case:** You would use UNION when listing all distinct rows from multiple tables or queries.

UNION ALL Operator

- The UNION ALL operator returns all the rows from both queries, including the duplicates.

```
SELECT ID, Name, Gender, Department FROM TableA  
UNION ALL  
SELECT ID, Name, Gender, Department FROM TableB
```

Result:

ID	Name	Gender	Department
1	Pranaya	Male	IT
2	Priyanka	Female	IT
3	Preety	Female	HR
3	Preety	Female	HR
2	Priyanka	Female	IT
3	Preety	Female	HR
4	Anurag	Male	IT

INTERSECT Operator

- The INTERSECT operator retrieves the common unique rows from the left and right queries. Notice the duplicates are removed.

**SELECT ID, Name, Gender, Department FROM TableA
INTERSECT
SELECT ID, Name, Gender, Department FROM TableB**

Result:

ID	Name	Gender	Department
2	Priyanka	Female	IT
3	Preety	Female	HR

INTERSECT Operator

- **Purpose:** The INTERSECT operator returns all rows common to both SELECT statements.
- **Distinct Values:** Like UNION and EXCEPT, INTERSECT also removes duplicates.
- **Use Case:** You would use INTERSECT when you need to find rows that are shared between two tables or queries.

EXCEPT Operator

- The EXCEPT operator will return unique rows from the left query that aren't in the right query's results.

```
SELECT ID, Name, Gender, Department FROM TableA
EXCEPT
SELECT ID, Name, Gender, Department FROM TableB
```

Result:

ID	Name	Gender	Department
1	Pranaya	Male	IT

DISTINCT

-- select the unique countries from the customers table

```
SELECT DISTINCT country  
FROM Customers;
```

Table: Customers

customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE



```
SELECT DISTINCT country  
FROM Customers;
```



country
USA
UK
UAE

COUNT

```
SELECT COUNT(DISTINCT country)  
FROM Customers;
```

COUNT

--returns the number of rows in the Customers table

```
SELECT COUNT(*)  
FROM Customers;
```


GROUPBY

-- select the item column and the count of order ids from the Orders table

-- group them by the item column

```
SELECT COUNT(order_id), item
```

```
FROM Orders
```

```
GROUP BY item;
```

Table: Customers

customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE



```
SELECT country, COUNT(*) AS number  
FROM Customers  
GROUP BY country;
```



country	number
UAE	1
UK	2
USA	2

ORDERBY

-- orders all rows from Customers in ascending order by country

SELECT *

FROM Customers

ORDER BY country;

Table: Customers

customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE

SELECT *
FROM Customers
ORDER BY first_name;

customer_id	first_name	last_name	age	country
5	Betty	Doe	28	UAE
3	David	Robinson	22	UK
1	John	Doe	31	USA
4	John	Reinhardt	25	UK
2	Robert	Luna	22	USA

ORDERBY ASC

-- orders all rows from Customers in ascending order by age

SELECT *

FROM Customers

ORDER BY age ASC;

ORDERBY DESC

-- order all rows from Customers in descending order by age

SELECT *

FROM Customers

ORDER BY age DESC;